

SOC Internship: Lab Task Report

Isolated Virtual Lab: Linux and Windows VM Connectivity

Field	Details
Intern Name	Amrita Pathak
Date	04 September 2026
Internship Track	SOC (Security Operations Center)

1. Objective

Build an isolated virtual lab using VMware Workstation, consisting of one Linux VM (Kali Linux) and one Windows VM (Windows 11). Configure both on the same virtual network so they can communicate, record each VM's IP address, verify connectivity with ping, and document the topology.

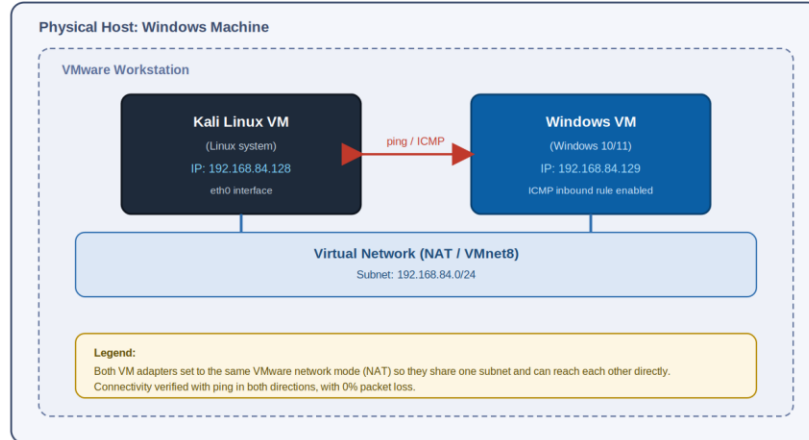
2. Tools/Commands Used

Tool	Purpose
VMware Workstation	Hypervisor used to create and run both VMs
Kali Linux (VM)	Linux system used in the lab
Windows 11 (VM)	Windows system used in the lab
ip a / ipconfig	Commands used to find each VM's IP address
ping	Command used to test connectivity between the two VMs
Windows Defender Firewall	Configured to allow inbound ICMP so ping would succeed

3. Network Topology

Both VMs were placed on the same VMware NAT network, allowing them to communicate directly while remaining isolated from any external production network.

Isolated VM Lab Topology

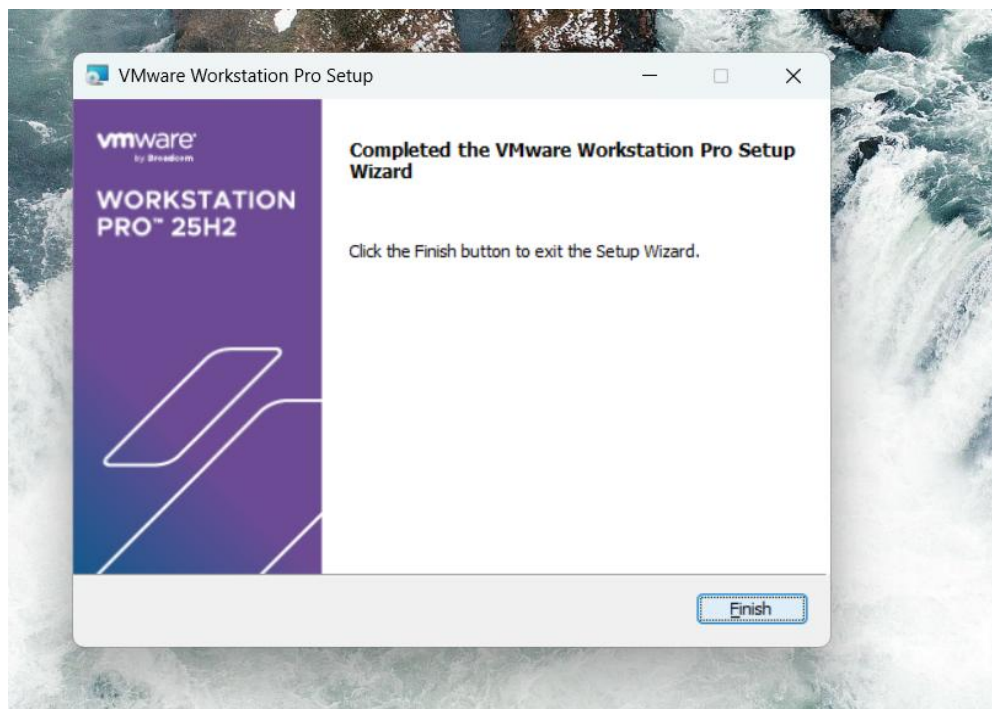


Lab topology: Kali Linux VM and Windows VM on a shared NAT network.

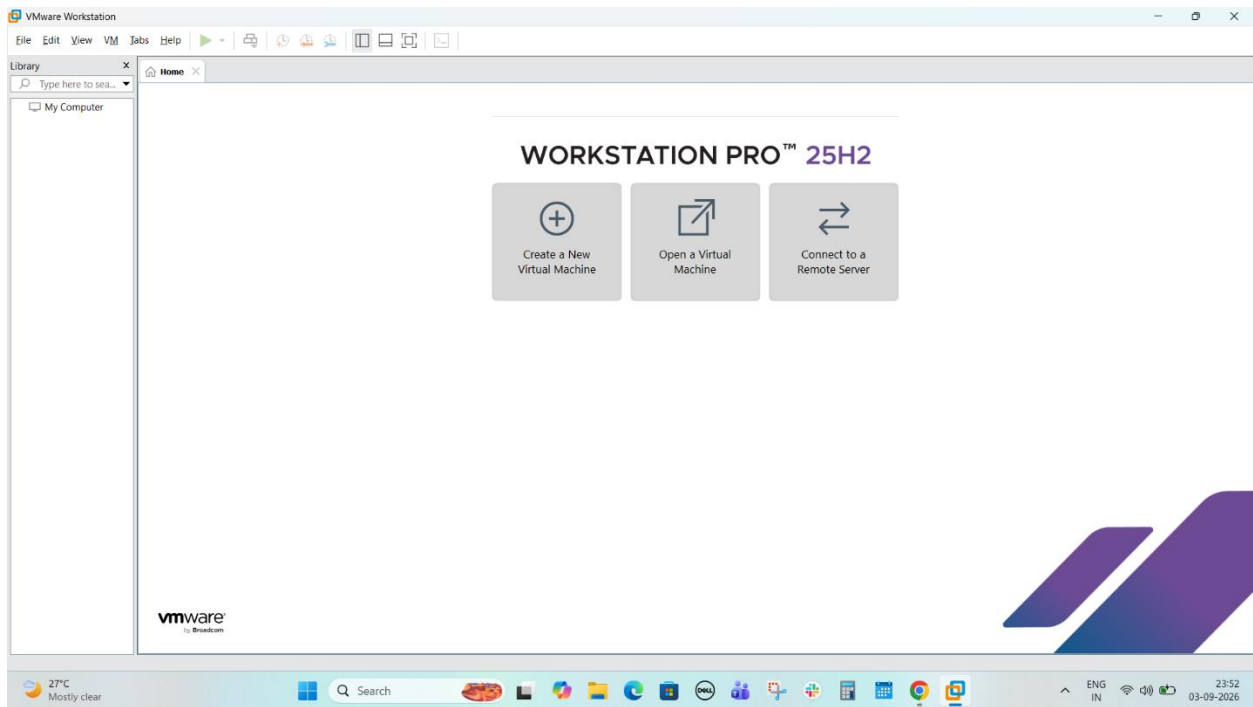
4. Methodology

4.1 Setting Up the VMs

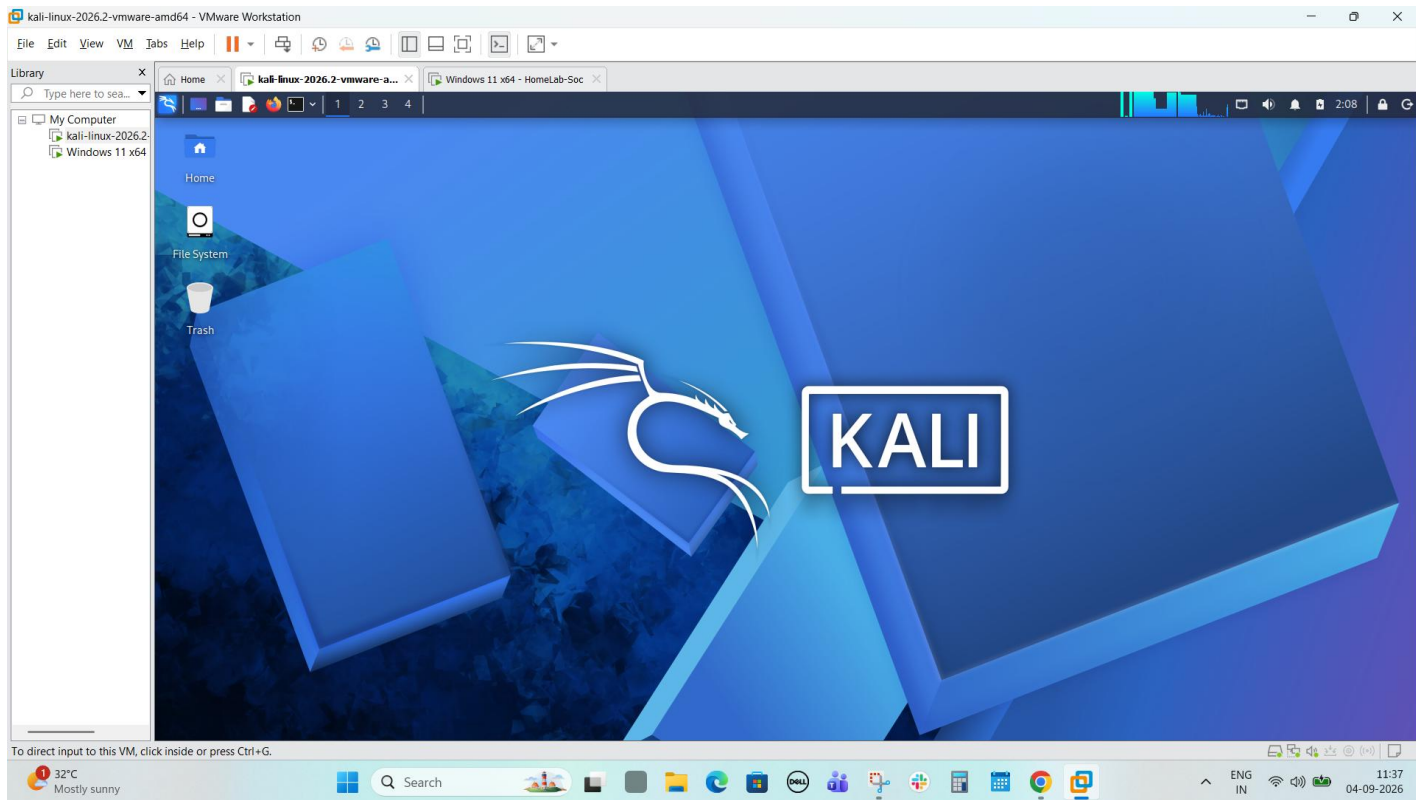
- Installed VMware Workstation Pro on the host machine.



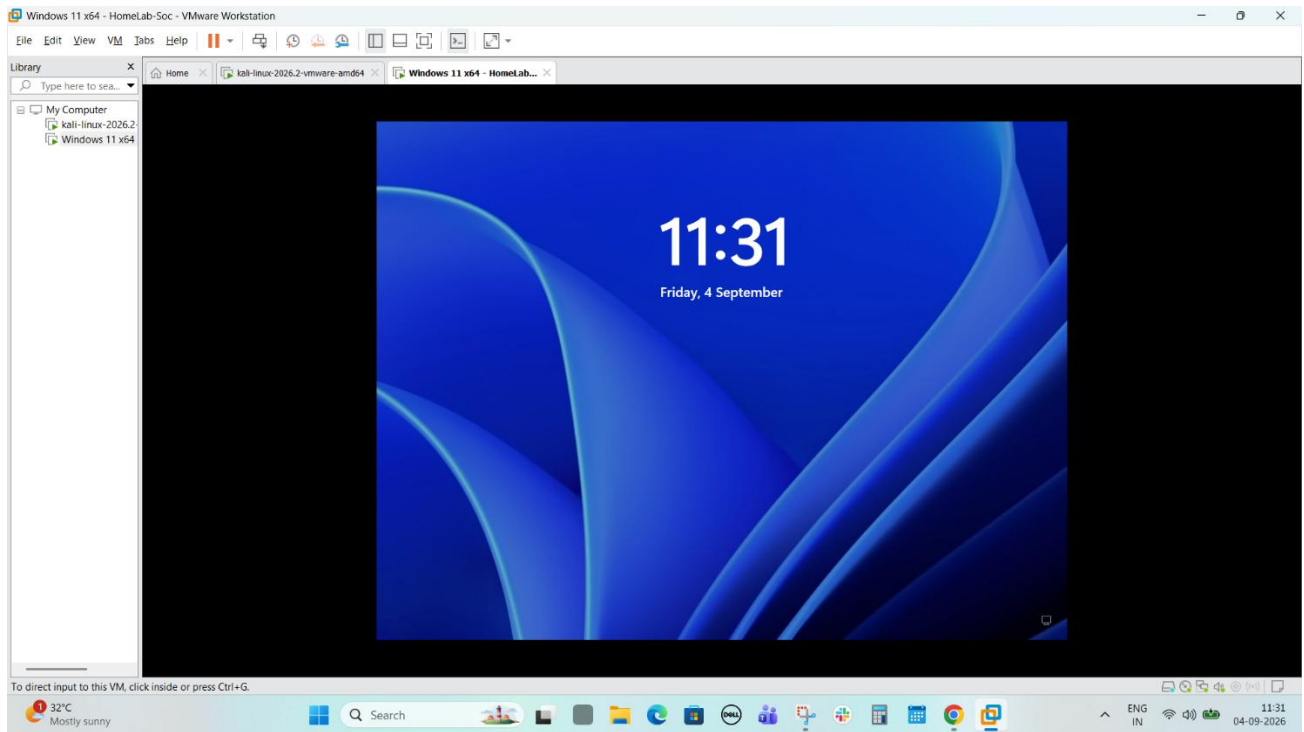
VMware Workstation Pro installed on the host machine.



- Set up a Kali Linux VM (pre-built appliance, imported and run directly).

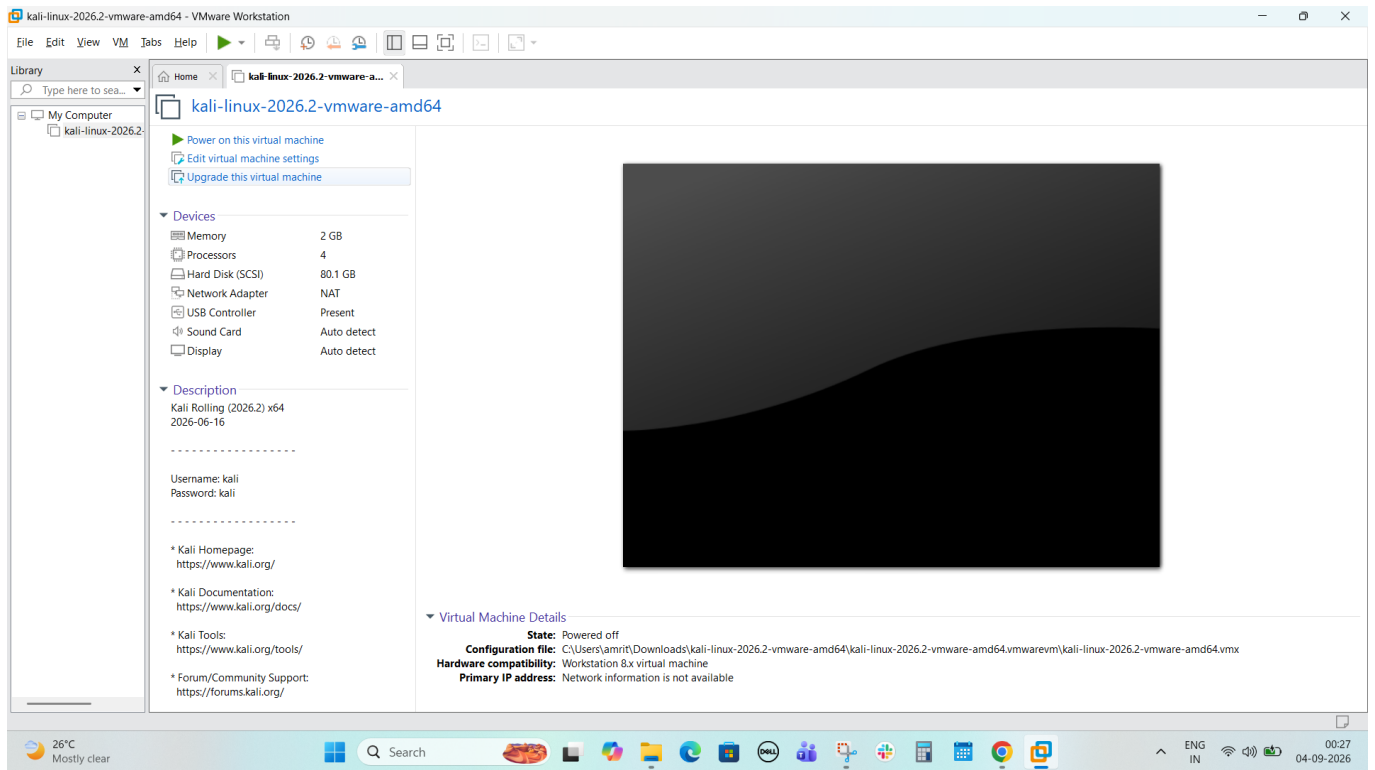


- Created a new Windows VM: downloaded the official Windows 11 ISO from Microsoft, then in VMware used File > New Virtual Machine, selected the ISO as the installer disc image, and let the wizard configure the VM (disk, memory, NAT network adapter by default).
- Completed the Windows install through the on-screen setup (language/edition, disk selection, automatic restarts) until it reached the desktop.

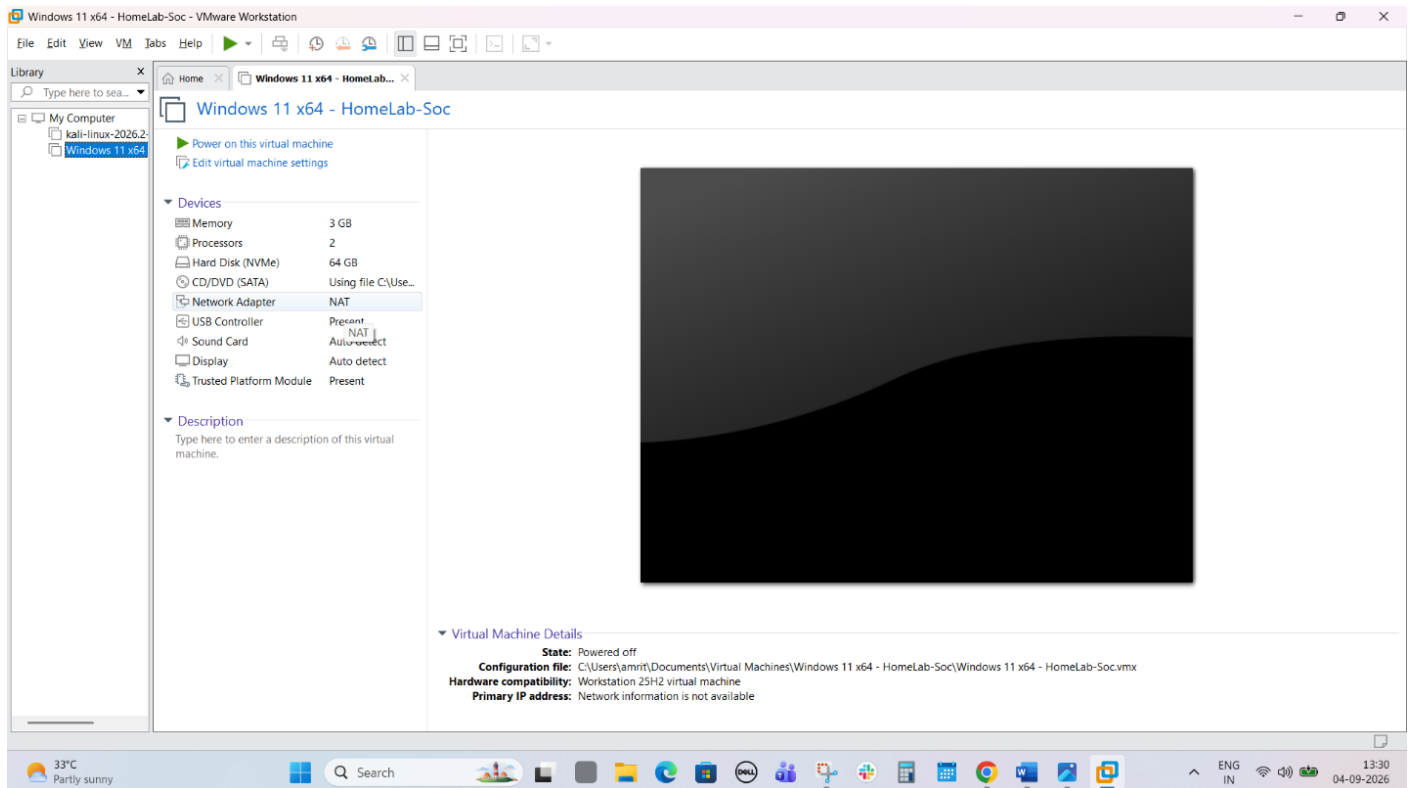


4.2 Configuring Networking (NAT)

- Opened each VM's Settings > Network Adapter and set both to NAT — the simplest option that puts both VMs on the same virtual subnet automatically.
- Confirmed the setting in each VM's configuration summary.



Kali Linux VM configuration — Network Adapter set to NAT.

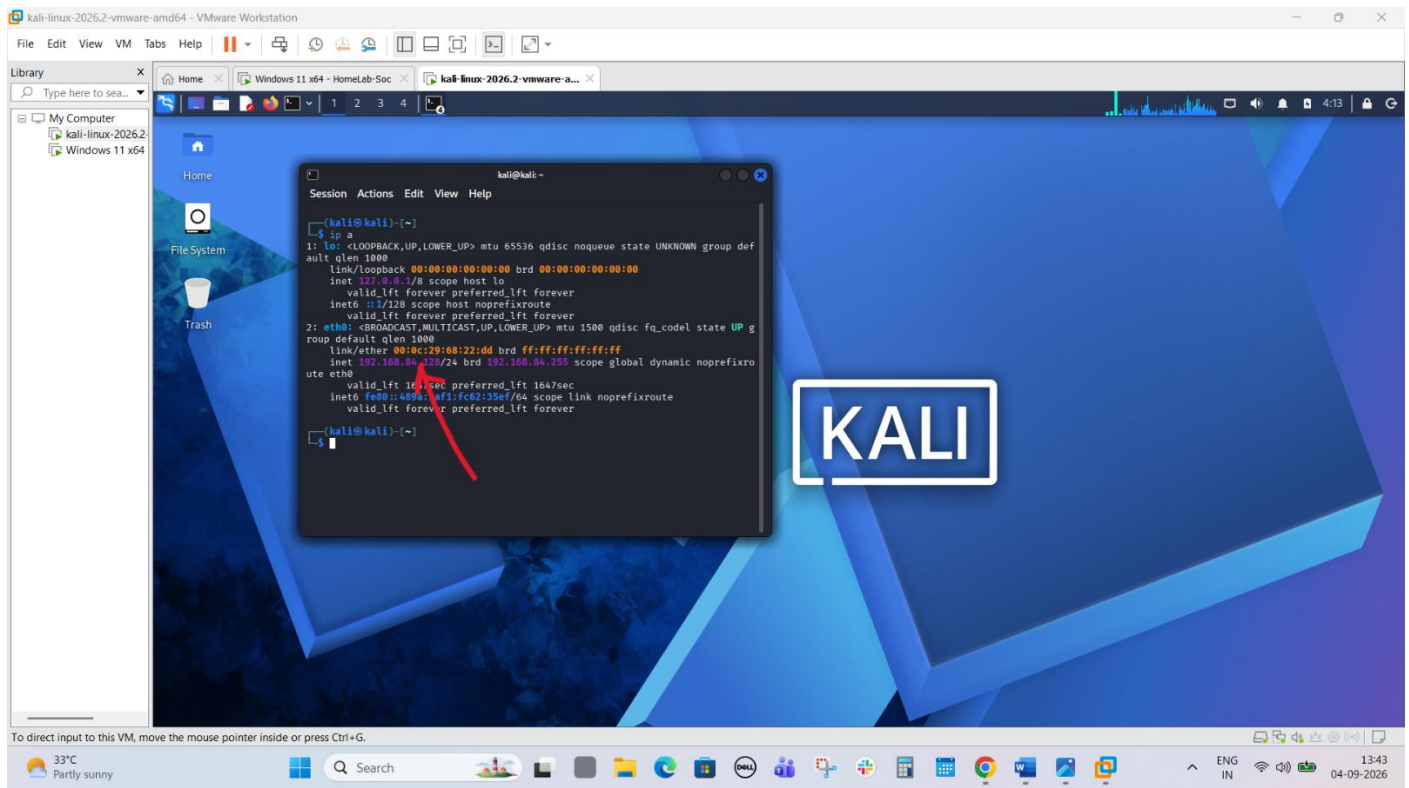


Windows VM configuration — Network Adapter set to NAT.

4.3 Recording IP Addresses

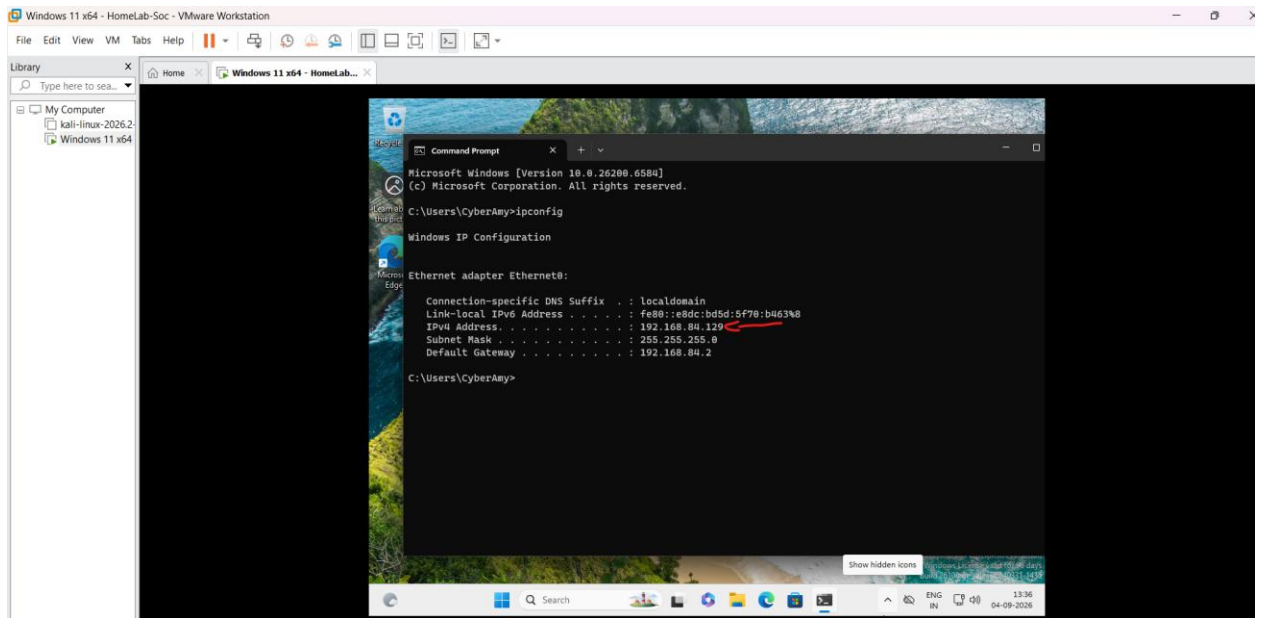
- On Kali: opened a terminal and ran `ip a`, noted the IPv4 address under `eth0`.

```
ip a
```



- On Windows: opened Command Prompt and ran `ipconfig`, noted the IPv4 address under the Ethernet adapter.

```
ipconfig
```



Confirmed both addresses were in the same /24 subnet (192.168.84.x), meaning the two VMs could reach each other directly.

4.4 Enabling ICMP Through Windows Firewall

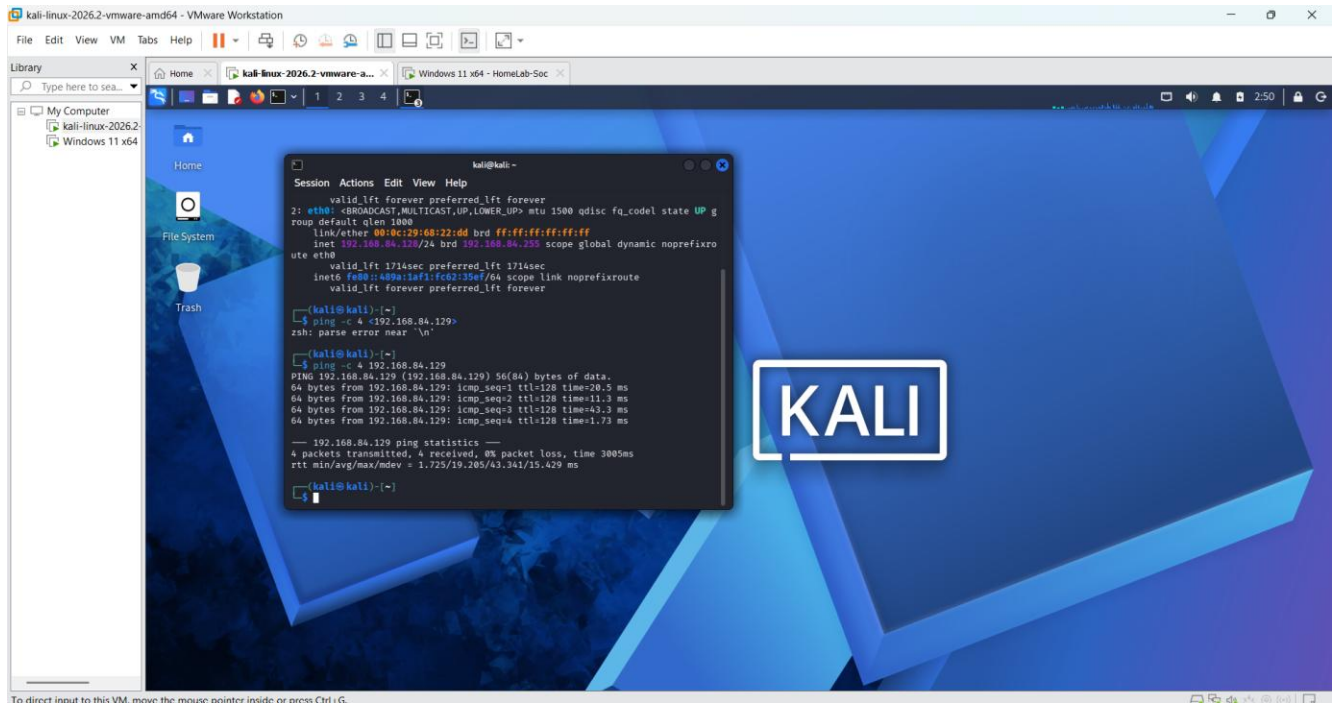
- Windows blocks incoming ping (ICMP) requests by default, so this step was required before the Kali-to-Windows ping would succeed.
- Opened an elevated (Administrator) PowerShell on the Windows VM, since the firewall change did not take effect from a standard, non-administrator prompt.
- Ran the following command to allow ICMP through for this isolated lab test:

```
netsh advfirewall set allprofiles state off
```

4.5 Testing Connectivity (Ping)

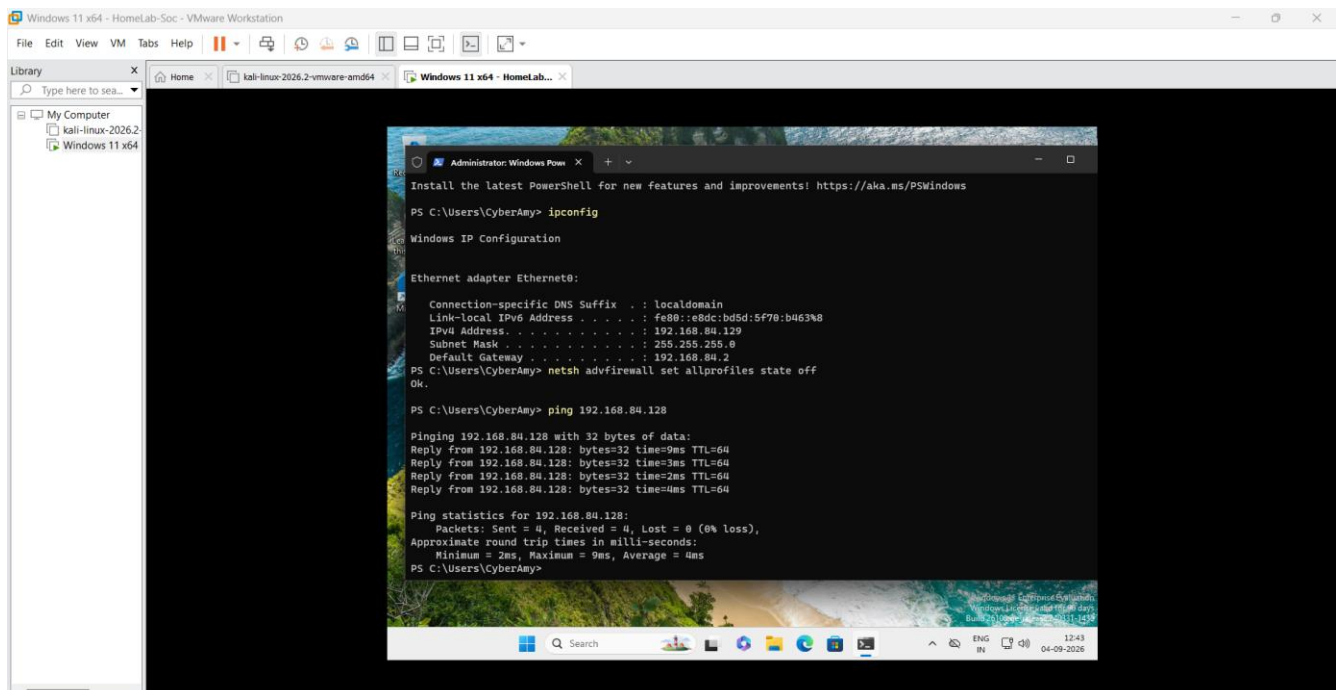
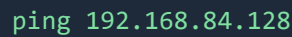
- From Kali, pinged the Windows VM's IP address:

```
ping -c 4 192.168.84.129
```



Kali Linux: 'ip a' showing its IP address, and a successful ping to the Windows VM.

- From Windows, pinged the Kali VM's IP address:



Windows: firewall command, 'ipconfig' showing its IP address, and a successful ping to the Kali VM.

- Both directions returned successful replies with 0% packet loss.

5. IP Address Summary

System	IP Address
Kali Linux VM	192.168.84.128
Windows VM	192.168.84.129

6. Conclusion

An isolated lab environment was successfully built with one Linux VM (Kali) and one Windows VM, both connected on the same NAT-based virtual network and confirmed to be on the same /24 subnet (192.168.84.0/24). Both VMs were configured with matching network settings, the Windows Firewall was adjusted to permit ICMP, and IP addresses were recorded for each system. Connectivity was verified using ping in both directions with 0% packet loss, confirming successful bidirectional communication between the two systems.